Solution Composition and Project Boundaries

Overview

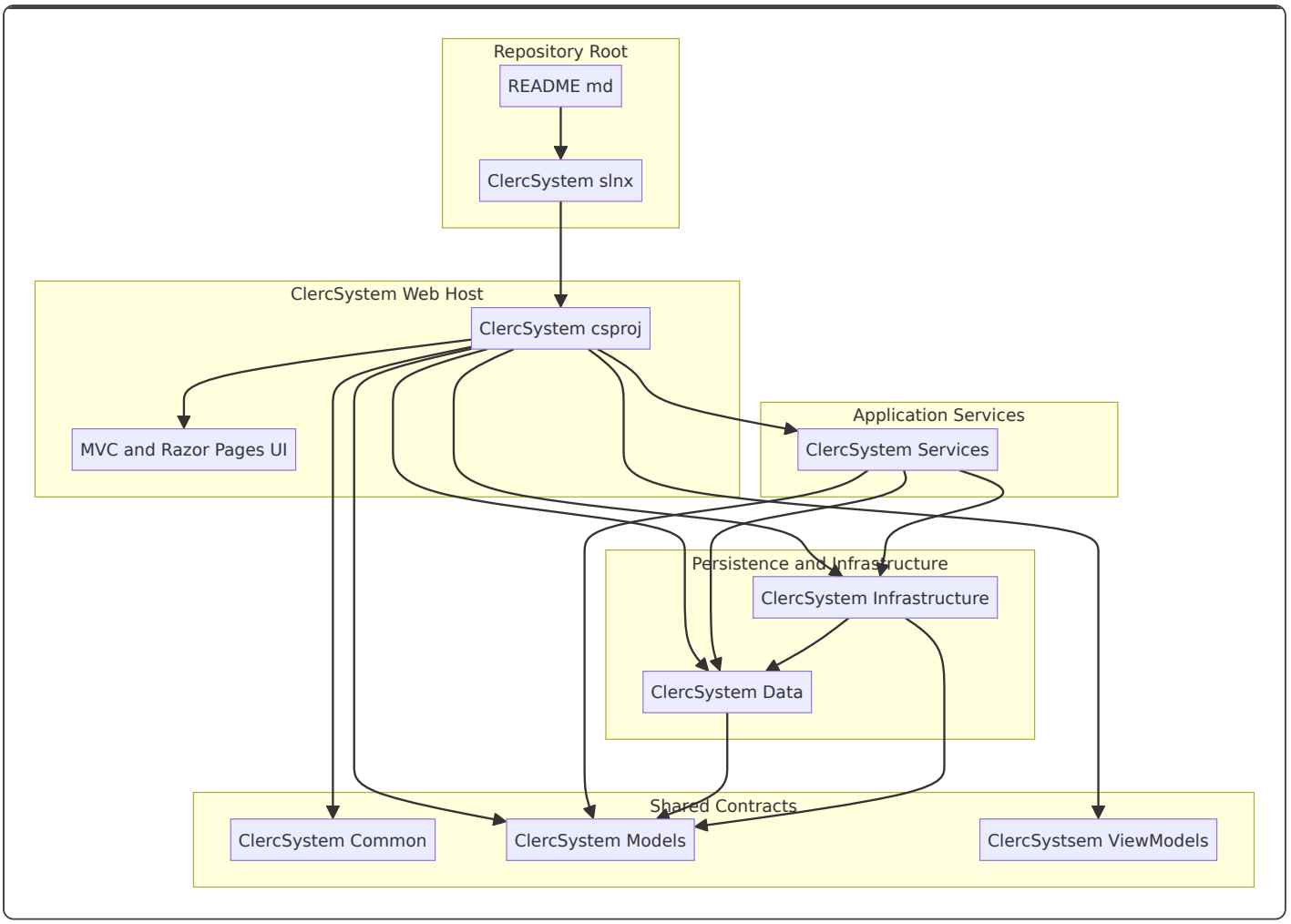
This repository is organized as a layered .NET solution where the web project is the runtime host and the other projects are referenced libraries that keep the UI, persistence, repositories, business services, and shared contracts separated.

The structure supports a clean composition model for ClercSystem: MVC and Razor Pages stay in the web project, EF Core persistence stays in the data boundary, repositories and supporting infrastructure remain isolated, use-case logic is grouped in services, shared models are reused across layers, and `ClercSystem.Common` centralizes validation constants used throughout the application.

Repository Layout

Path	Kind	Boundary Role	Responsibility
`README.md`	Documentation	Repository entry point	Describes the solution at a high level and anchors the project structure.
`Clerk System/Clerc System/` `ClercSystem.slnx`	Solution	Composition root for the solution	Groups the projects into a single buildable solution.
`Clerk System/ClercSystem/ClercSystem/` `ClercSystem.csproj`	Web project	Runtime host and presentation boundary	Hosts the ASP.NET Core MVC and Razor Pages application and references the supporting layers.
`ClercSystem.Common`	Class library	Shared support boundary	Hosts validation constants shared by the rest of the solution.
`ClercSystem.Models`	Class library	Shared model boundary	Carries the models used across the application layers.
`ClercSystem.Data`	Class library	Persistence boundary	Contains EF Core persistence concerns.
`ClercSystem.Infrastructure`	Class library	Infrastructure boundary	Holds repository-oriented infrastructure used by the application.
`ClercSystem.Services`	Class library	Application service boundary	Contains use-case services that coordinate application behavior.
`ClercSystsem.ViewModels`	Class library	UI contract boundary	Contains view-model types consumed by the MVC and Razor Pages UI.

Architecture Overview



The diagram reflects the project boundary strategy used in the solution: the web host composes the app, while the rest of the solution is split into reusable layers that keep concerns isolated. The web project is the only runtime-facing entry point; the remaining projects are library dependencies that shape the application's internal architecture.

Solution Composition

`ClercSystem.slnx`

`Clerk System/ClercSystem/ClercSystem.slnx`

The solution file groups the full ClercSystem codebase into a single workspace. It is the top-level composition artifact that brings together the web host, shared libraries, persistence, infrastructure, services, and view-model projects.

Aspect	Description
Purpose	Defines the solution boundary and project set.
Composition role	Collects all layers into one build graph.
Runtime role	None directly; it supports the web project that runs the application.

`ClercSystem.csproj`

`Clerk System/ClercSystem/ClercSystem/ClercSystem.csproj`

This is the executable web project. It is the boundary where the MVC and Razor Pages UI meets the rest of the application, and it depends on the supporting projects that provide shared models, persistence, repositories, services, and UI-facing view models.

Aspect	Description
Purpose	Hosts the ASP.NET Core web application.
Boundary	Presentation and runtime composition root.
Depends on	`ClercSystem.Common` , `ClercSystem.Models` , `ClercSystem.Data` , `ClercSystem.Infrastructure` , `ClercSystem.Services` , `ClercSystsem.ViewModels` .
Responsibility	Keeps the UI thin and delegates application behavior to lower layers.

Project Boundaries

`ClercSystem.Common`

> **NOTE**
> The project graph is centered on `ClercSystem.csproj` as the runtime host. The other projects are referenced as libraries so the solution can separate UI, persistence, repositories, and use-case logic without collapsing those concerns into the web layer.

Shared validation constants

`ClercSystem.Common` is the shared constants boundary. Its role is to host validation constants that can be reused across the solution so UI, services, and other layers can rely on the same validation values.

Boundary concern	Description
Validation constants	Centralizes shared validation values.
Cross-layer reuse	Provides constants that can be consumed by more than one project.
Coupling control	Keeps validation values out of UI and data code.

`ClercSystem.Models`

Shared model boundary

`ClercSystem.Models` is the shared model project that provides application-wide model types used by the other layers. It forms the contract layer that the service, persistence, and UI boundaries can share.

Boundary concern	Description
Shared types	Supplies models used across projects.
Contract stability	Keeps data-shape definitions outside the web host.
Layer separation	Avoids duplicating model definitions in UI or persistence code.

`ClercSystem.Data`

EF Core persistence boundary

`ClercSystem.Data` contains the EF Core persistence concerns. It is the layer that keeps database access separate from the UI and from the use-case services.

Boundary concern	Description
Persistence	Owns EF Core data access responsibilities.
Database interaction	Serves as the database-facing layer for the application.
Separation	Prevents persistence code from leaking into MVC or Razor Pages.

`ClercSystem.Infrastructure`

Repository boundary

`ClercSystem.Infrastructure` holds the repository-oriented infrastructure used by the application. It sits beside the EF Core persistence layer and keeps repository concerns distinct from both the UI and the use-case services.

Boundary concern	Description
Repository support	Houses repository-oriented implementation details.
Infrastructure concerns	Keeps lower-level support code away from the web layer.
Layer isolation	Helps preserve the separation between persistence and business use cases.

`ClercSystem.Services`

Use-case service boundary

`ClercSystem.Services` contains the application services that coordinate business use cases. It is the layer that the web project should call when it needs application behavior rather than direct persistence access.

Boundary concern	Description
Use cases	Groups application behavior into service-level operations.
Orchestration	Coordinates work across models, repositories, and persistence.
Presentation independence	Allows the UI to depend on application services instead of data code.

`ClercSystsem.ViewModels`

UI contract boundary

`ClercSystsem.ViewModels` contains the view models used by the web UI. These types shape data for MVC views and Razor Pages without forcing the UI to depend directly on persistence models.

Boundary concern	Description
UI shaping	Provides data shapes tailored for pages and views.
Presentation isolation	Separates UI contracts from persistence concerns.
Dependency direction	Lets the web project consume page-specific models cleanly.

Runtime Composition

The runtime is assembled through the web project, which pulls together the supporting projects during application startup. That arrangement keeps the executable surface in one place while allowing the rest of the solution to remain class libraries.

At runtime, the dependency direction is intentionally one-way from the web host into the supporting layers. The UI consumes view models and services; services coordinate application use cases; infrastructure and data handle persistence concerns; and `ClercSystem.Common` supplies shared validation constants that can be reused wherever validation rules are enforced.

Dependency Graph

From	To	Relationship
`ClercSystem.csproj`	`ClercSystem.Common`	Shares validation constants with the web layer.
`ClercSystem.csproj`	`ClercSystem.Models`	Uses shared model types in the web host.
`ClercSystem.csproj`	`ClercSystem.Data`	Includes EF Core persistence dependencies in the host graph.
`ClercSystem.csproj`	`ClercSystem.Infrastructure`	Consumes repository infrastructure from the application host.
`ClercSystem.csproj`	`ClercSystem.Services`	Calls use-case services from UI code.
`ClercSystem.csproj`	`ClercSystsem.ViewModels`	Uses UI-facing models for pages and views.
`ClercSystem.Services`	`ClercSystem.Models`	Coordinates application logic around shared models.
`ClercSystem.Services`	`ClercSystem.Data`	Integrates with EF Core persistence.
`ClercSystem.Services`	`ClercSystem.Infrastructure`	Uses repository infrastructure to perform work.
`ClercSystem.Data`	`ClercSystem.Models`	Stores and maps shared model types.
`ClercSystem.Infrastructure`	`ClercSystem.Data`	Builds on persistence-layer services and data access.
`ClercSystem.Infrastructure`	`ClercSystem.Models`	Uses shared model types in repository code.

Separation of Concerns

The solution is split so that each project owns a narrow boundary:

- **MVC and Razor Pages UI** live in the web project.
- **EF Core persistence** lives in `ClercSystem.Data`.
- **Repositories and supporting infrastructure** live in `ClercSystem.Infrastructure`.
- **Use-case services** live in `ClercSystem.Services`.

- **Shared models** live in `ClercSystem.Models`.
- **Validation constants** live in `ClercSystem.Common`.
- **UI-specific shapes** live in `ClercSystsem.ViewModels`.

That split keeps presentation concerns away from database code, keeps persistence concerns away from page rendering, and keeps application logic out of the UI project.

Runtime Boundary Summary

Boundary	What it owns	What it keeps out
Web host	MVC and Razor Pages execution	Direct persistence logic
Services	Application use cases	View rendering details
Infrastructure	Repository support	Page models and UI behavior
Data	EF Core persistence	UI-specific contracts
Models	Shared data shapes	Presentation-only state
Common	Validation constants	Layer-specific duplication
ViewModels	Page and view contracts	Database implementation details

Key Classes Reference

Class	Responsibility
`README.md`	Repository-level documentation and orientation.
`ClercSystem.slnx`	Solution composition and project grouping.
`ClercSystem.csproj`	Web host and runtime composition root.
`ClercSystem.Common`	Shared validation constants.
`ClercSystem.Models`	Shared application models.
`ClercSystem.Data`	EF Core persistence boundary.
`ClercSystem.Infrastructure`	Repository and infrastructure boundary.
`ClercSystem.Services`	Use-case service boundary.
`ClercSystsem.ViewModels`	UI-facing view-model boundary.